ENSF 380

TOPIC 18: INHERITANCE

Inheritance

A type of relationship between classes

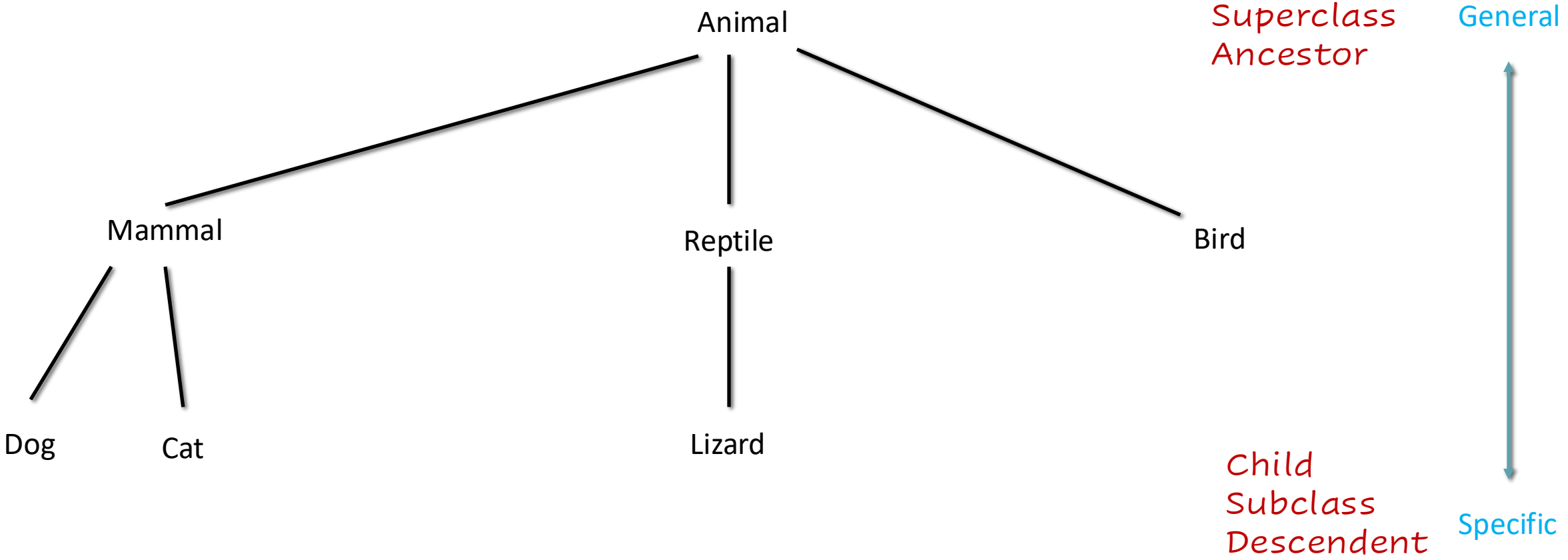
- Describes the "is a" hierarchy
- Common variables and behavior are generalized
- Specialized variables and behavior extend the generalization

The derivative class is a "superclass" or "parent" or "ancestor"

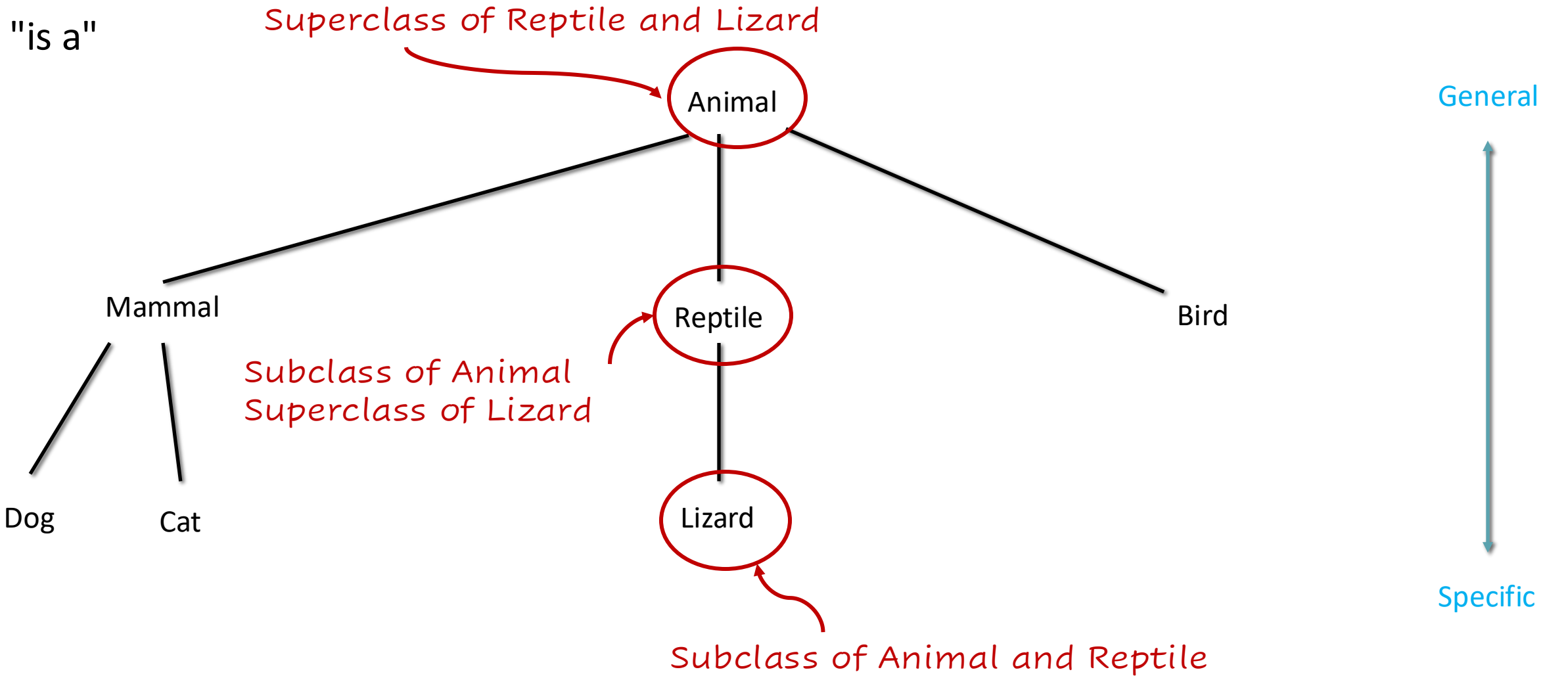
The derived class is a "subclass" or "child" or "descendent"

Hierarchy

"is a"

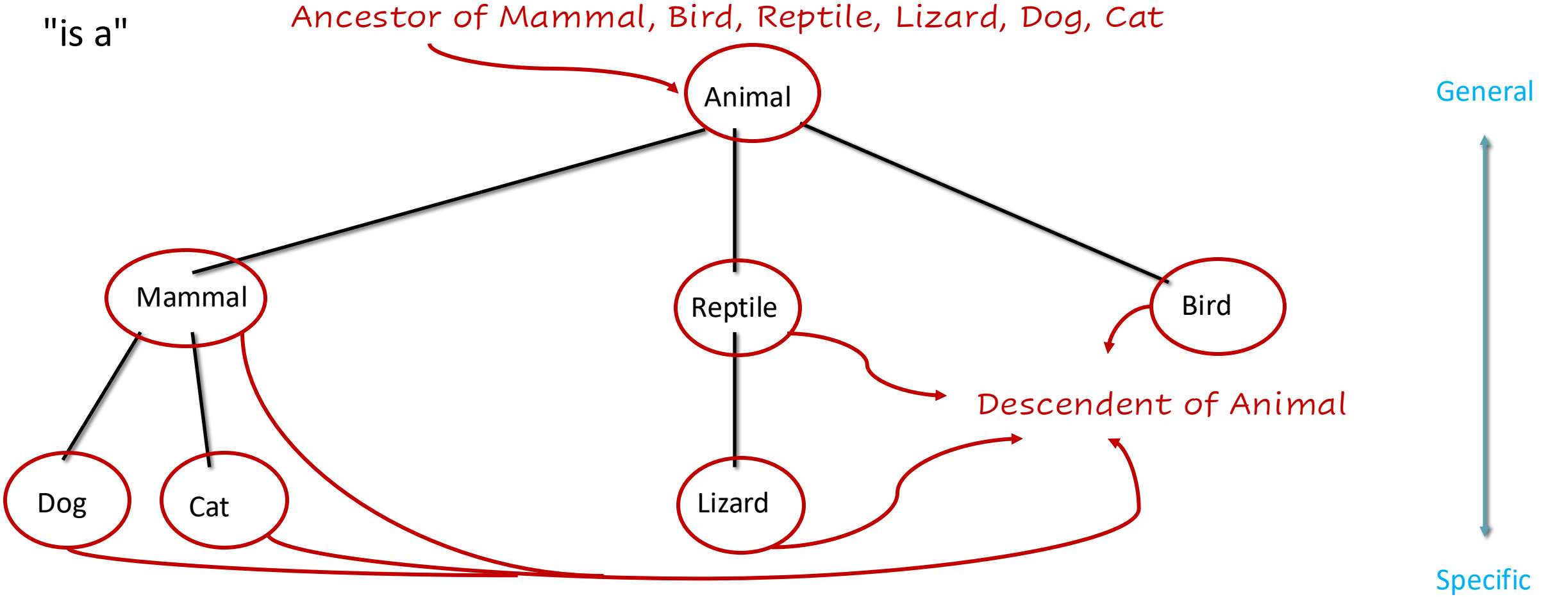


Hierarchy


"is a"
Superclass of Reptile and Lizard
Animal
Mammal
Reptile
Bird
Dog
Cat
Lizard
Subclass of Animal
Superclass of Lizard
Subclass of Animal and Reptile
General
Specific

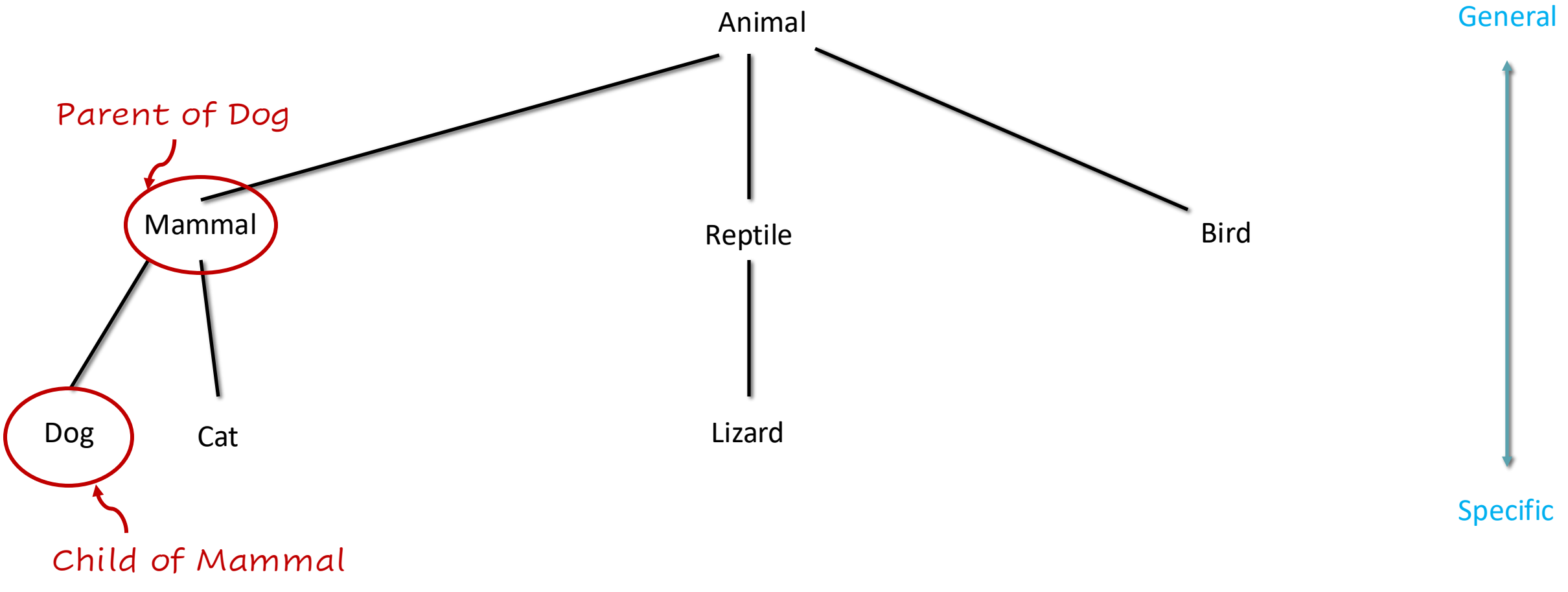
Hierarchy

"is a"

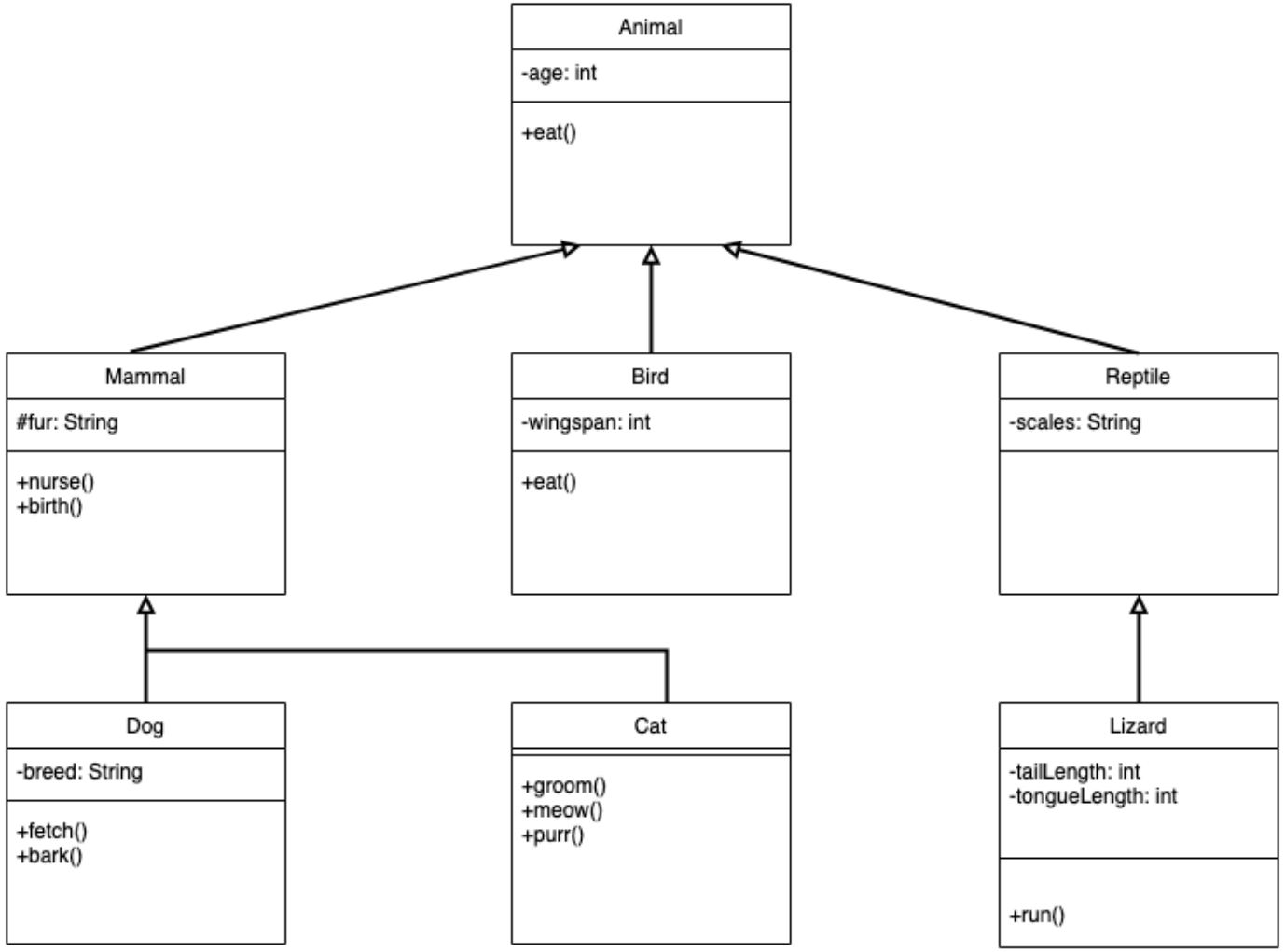


Hierarchy

"is a"



Inheritance



Inheritance

01_Reptile

```
class Reptile extends Animal {
        private String scales;

        public Reptile(int age) {
                super(age);
        }

        public Reptile(int age, String scales) {
                super(age);
                setScales(scales);
        }

        public String getScales() {
                return this.scales;
        }

        public void setScales(String scales) {
                this.scales = scales;
        }
}
```

Inheritance

01_Reptile

```
public static void  main(String[] args) {
        Reptile chameleon = new Reptile(2, "multi-hued");
        Reptile gecko = new Reptile(1);


        System.out.println("Chameleons are " +
        chameleon.getScales() + " and " +
        "this one is " + chameleon.getAge() +
        " years old.");


        System.out.println("This gecko is " +
        gecko.getAge() + " years old and is" +
        " enjoying a meal.");
        gecko.eat();
}
```

Explicit and Implicit Inheritance

Explicit Subclassing

```
class Animal extends Object {


}
```

Implicit Subclassing

```
class Animal {


}
```

Access Modifiers

Modifier	Visibility			
	Class	Package	Subclass	World
public	✓	✓	✓	✓
protected	✓	✓	✓	
default	✓	✓		
private	✓			

Exercise 18.1

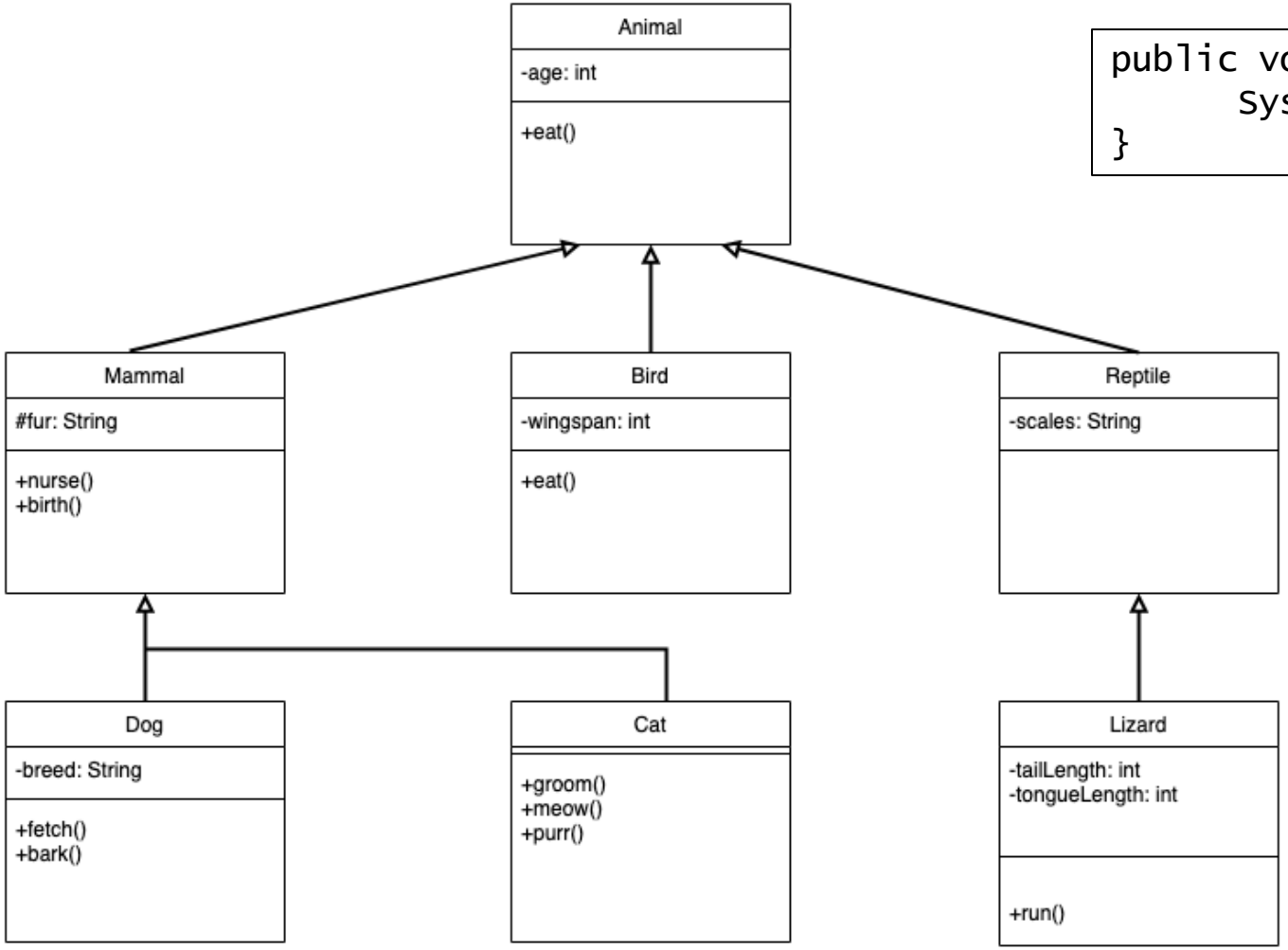
TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Exercise 18.2

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Overriding

03_Bird




```
public void eat() {
	System.out.println("Peck, peck, peck!");
}
```

Annotations

03_Bird

Begin with @

By convention, on its own line

Javadoc comments start with lowercase, regular annotations start with a capital letter

Metadata

- Information for compilation, processing, etc.

Javadoc examples

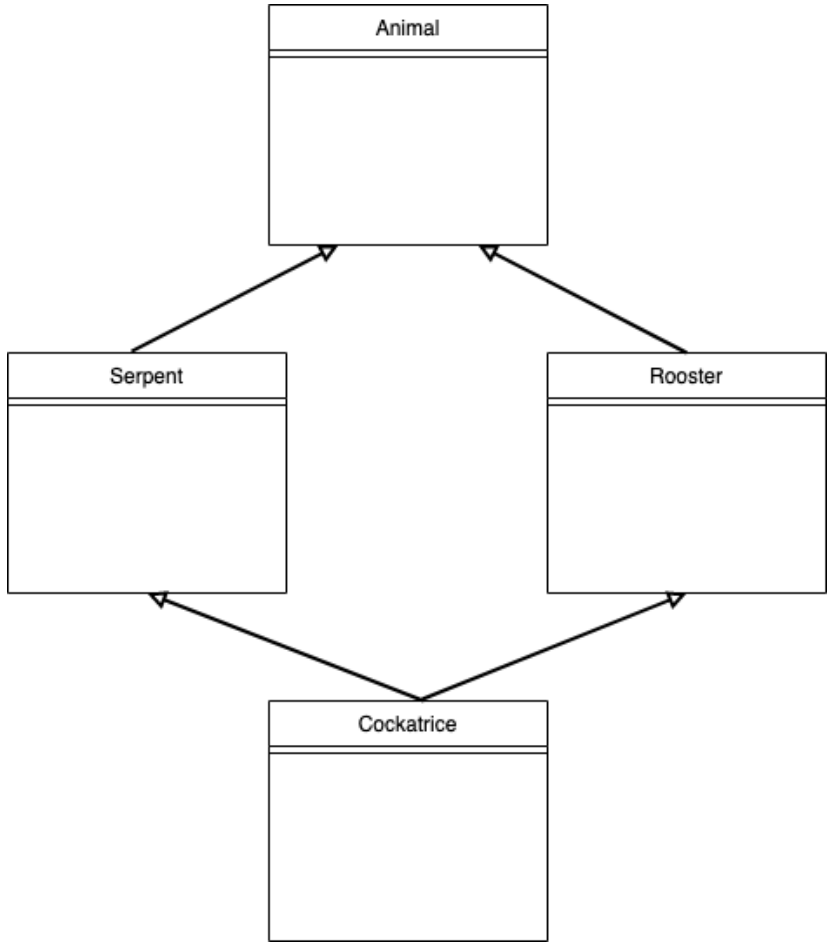
- @version
- @param

Useful annotation

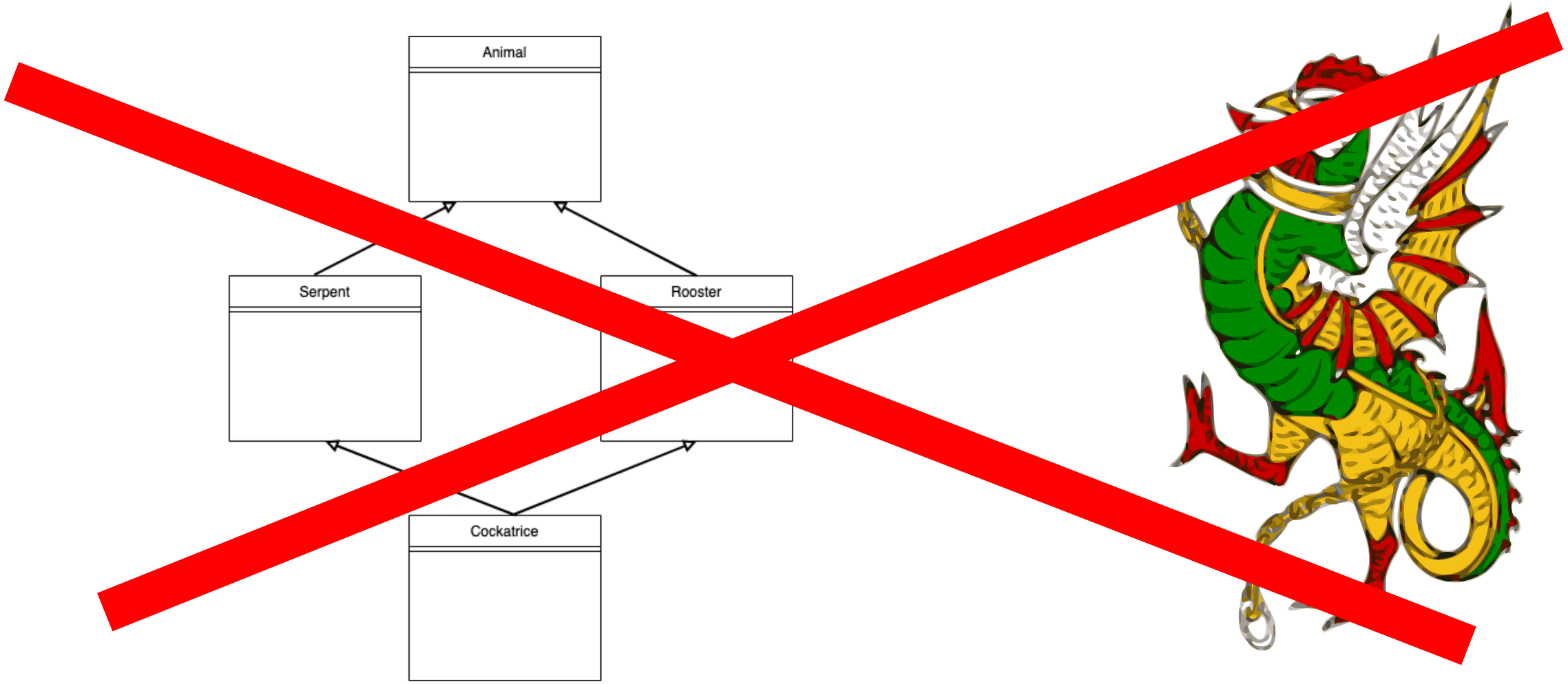
- @Override

```
@Override
String overrideMethod() { ...}
```

Multiple Inheritance



Multiple Inheritance

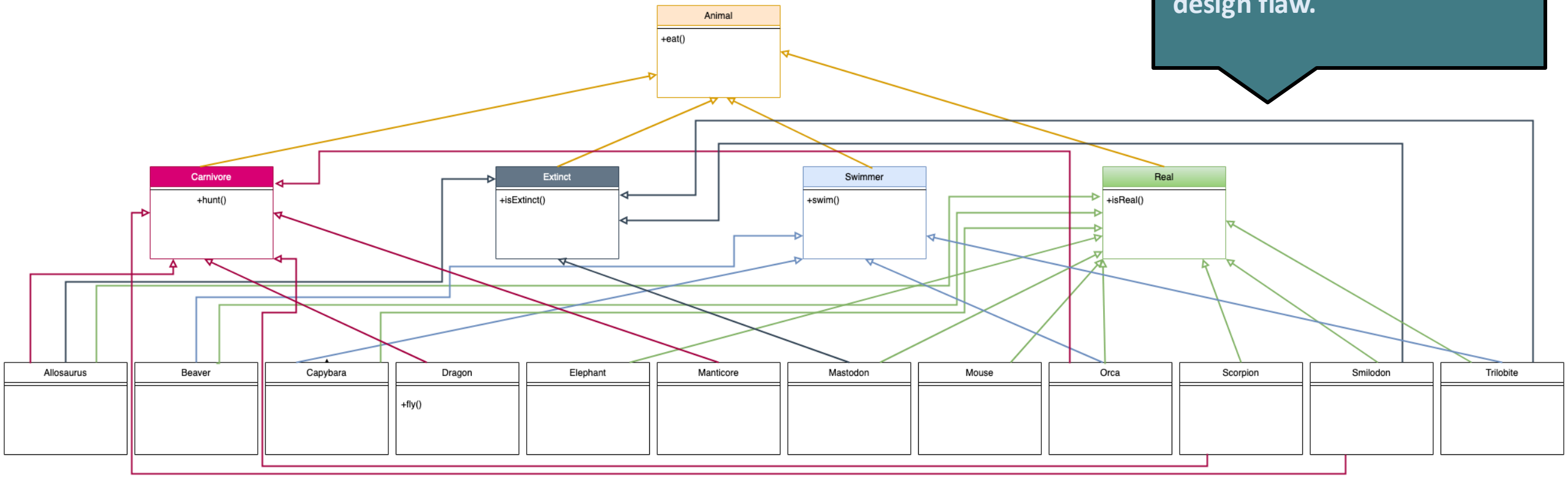


Exercise 18.3

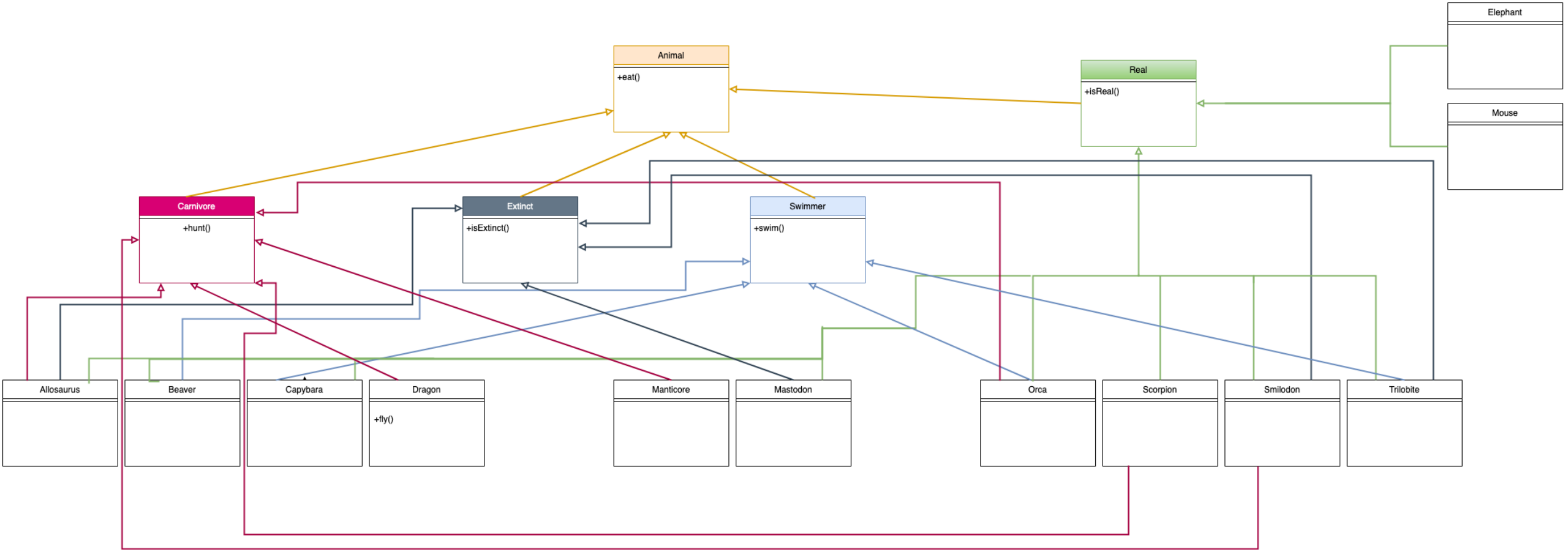
TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Exercise Solution

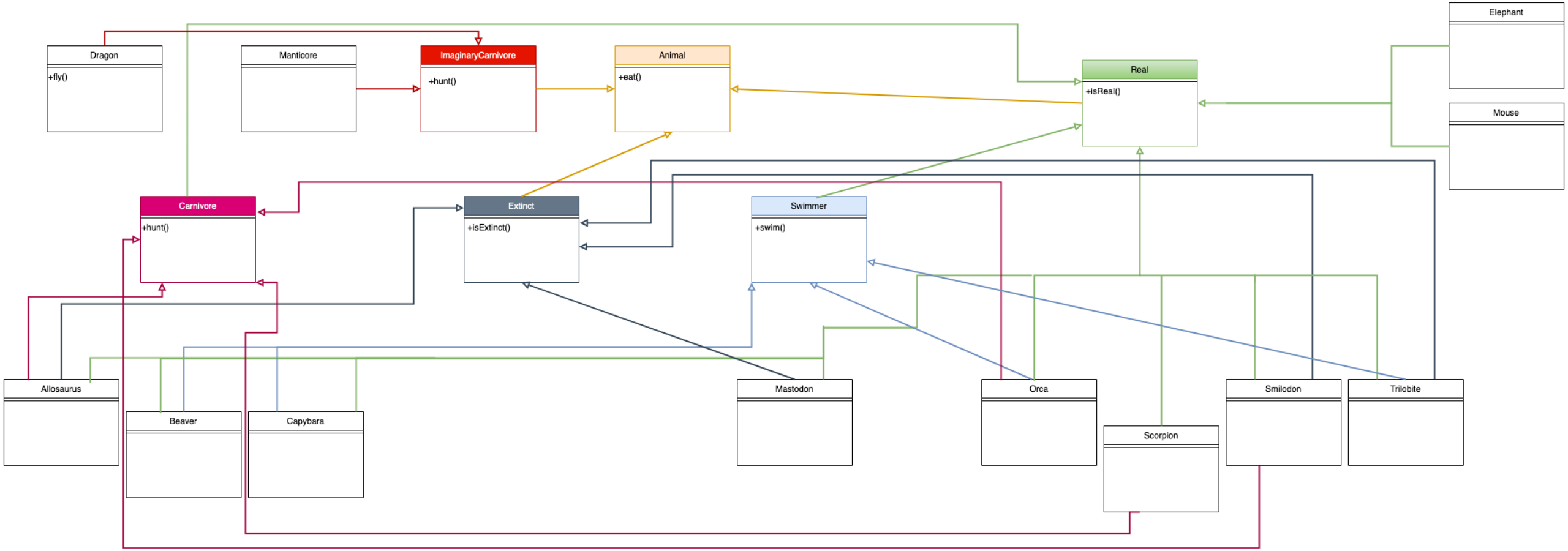
Tip: The colourful lines shown here are not part of the diagram and are only for clarity. If your UML is this complex, there is probably a design flaw.



Exercise Solution

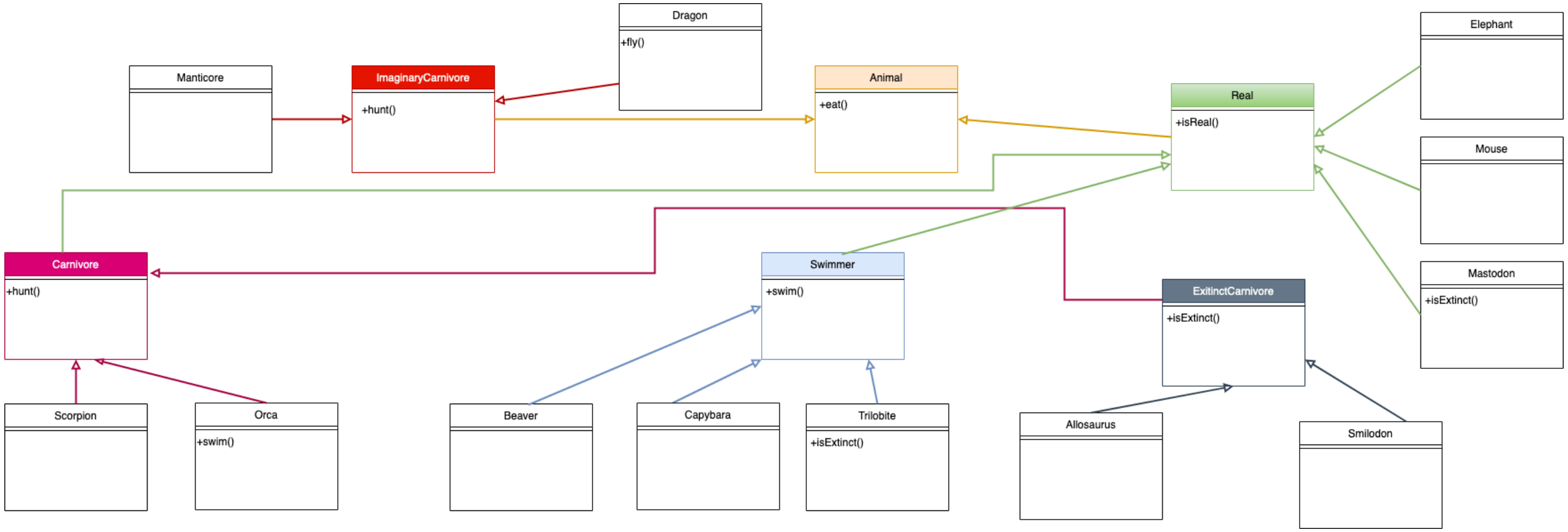


Exercise Solution



Exercise Solution

04_Bestiary



Abstract Classes

05_AbstractClasses

An abstract class cannot be instantiated

- Provides common structure and behavior
- Adds necessary parts to make a useful class
- May contain one or more abstract methods

Classes which are not abstract are concrete

```
abstract class Swimmer extends Real {
        public void swim() {
                System.out.println("Method swim is supplied by Swimmer");
        }
}
```

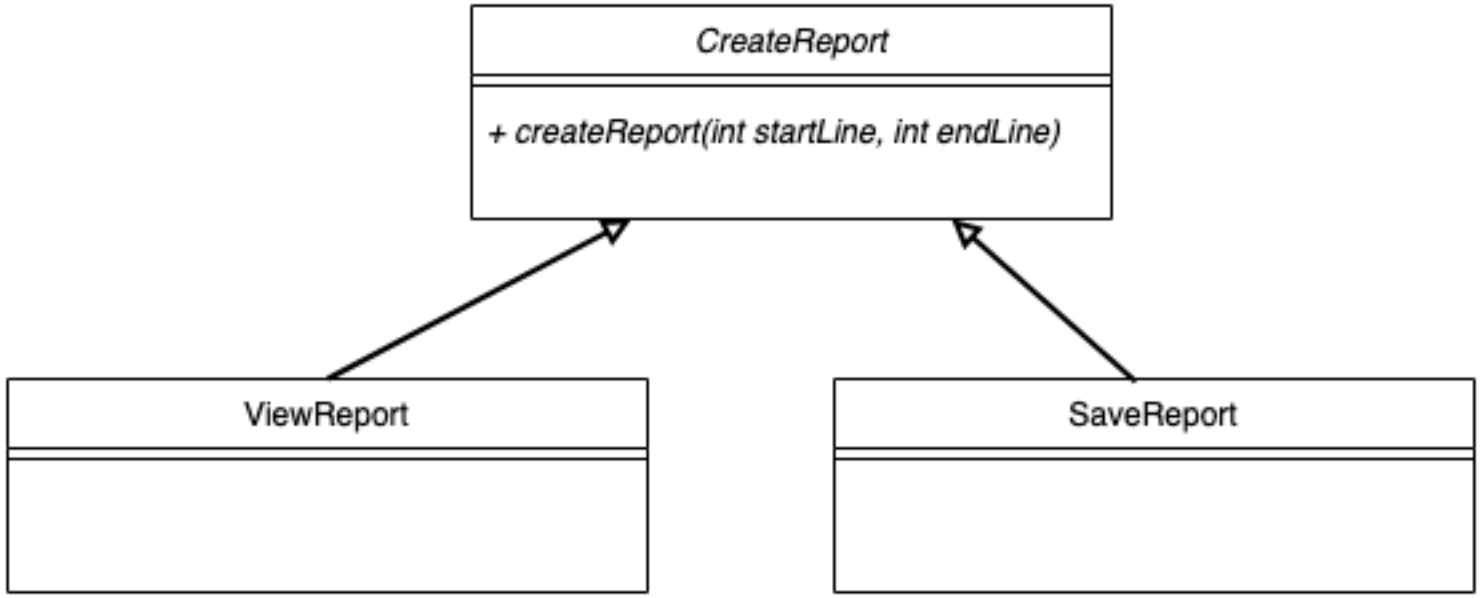
Abstract Classes

05_AbstractClasses

```
abstract class Swimmer extends Real {
        public void swim() {
                System.out.println("Method swim is supplied by Swimmer");
        }
        public abstract void breathe();
}

class Trilobite extends Swimmer {
        @Override
        public void breathe() {
                System.out.println("Method breathe is supplied by Trilobite");
        }
}
```

Abstract Classes



Summary: Abstract versus Concrete

Abstract Class	Concrete Class
Can have abstract methods	Can not have abstract methods
Can have concrete methods	Can only have concrete methods
Cannot be instantiated	Can be instantiated
Must be inherited from to be used	Can be inherited from
Is indicated with the 'abstract' keyword	Is indicated by the absence of the 'abstract' keyword
Is written in italics in UML	Is not written in italics in UML

Abstract Method	Concrete Method
Must be overridden in concrete child class	Can be overridden by child class
Is indicated with the 'abstract' keyword	Is indicated by the absence of the 'abstract' keyword
Is written in italics in UML	Is not written in italics in UML

Polymorphism

Static Polymorphism (e.g., overloading)

Dynamic Polymorphism (e.g., overriding)

- Different objects accessed through the same interface, but different implementation
- Works when classes are related in the inheritance tree (common superclass)
- Subclass redefines the method to specialize the behavior

Final Classes

05_AbstractClasses

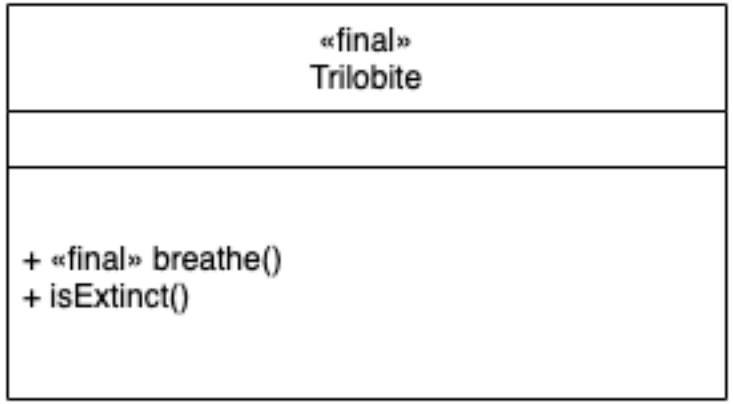
A final class cannot be extended

A final method cannot be overridden

```
final class Trilobite extends Swimmer {
        public void isExtinct() {
                System.out.println("Method isExtinct is supplied by Trilobite");
        }

        @Override
        final public void breathe() {
                System.out.println("Method breathe is supplied by Trilobite");
        }
}
```

Stereotype



Exercise 18.4

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Summary: Knowledge

Inheritance

- explicit and implicit
- abstract classes and methods
- final classes and methods
- multiple inheritance

Private and protected access modifiers

Polymorphism

- static and dynamic

Annotations

Summary: Skills

Represent inheritance in UML

- abstract classes and methods
- final classes and methods

Use stereotypes in UML

Implement inheritance in Java

- abstract keyword
- final keyword
- overriding a method

Additional Information

Related reading

- Volume 1, Chapter 5.1: Classes, Superclasses, and Subclasses
- Volume 1, Chapter 5.2: Object: The Cosmic Superclass